

LMA Group Project 2026

In this project your team will pick a real problem from the domain of your choice, build a small AI system that tackles it, and show through careful evaluation whether your idea works.

1. What you are expected to do

Your task is not simply to build another chatbot or RAG application. You should identify a concrete problem in a domain, propose a novel approach, build a small prototype, and evaluate whether the idea helps.

The project should be research oriented. Your final submission should contain a clear research question, a proposed method or system, an evaluation setup, and evidence that supports or challenges your hypothesis.

The domain gives you the problem, the data and the constraints. Your research contribution should come from how you combine and study the capabilities described in the next section.

2. The five capabilities

Your project should combine the following five capabilities in varying amounts. Which ones you lean on depends on your problem. A project may use all five, but it should not rest on only one.

1. **Fine-tuning:** adapting a small or open model to your domain task using lightweight methods such as LoRA or QLoRA. For example, you might show that a tuned small model matches a much larger prompted model on a narrow task.
2. **RAG:** grounding the system's answers in a domain corpus such as documents, records, runbooks, statutes or course material. Contributions could include better retrieval, evidence checking, abstention or citation faithfulness.
3. **Reasoning:** multi-step inference such as planning, hypothesis testing, decomposition, verification or self-critique. You should analyse the intermediate steps, not only the final answer.
4. **Agentic abilities:** the system decides what to do next over many steps, using planning, memory, state tracking, multiple cooperating agents or hand-off to a human.
5. **Tool calling:** the model invokes external functions such as search, databases, APIs, code execution or domain systems. You should show that it picks the right tools, passes valid arguments, recovers from errors and never takes unsafe actions.

For each capability you use, explain in your proposal why your problem needs it and how you will measure whether it works. Adding a capability only because it appears on this list does not count.

3. Choosing your domain and topic

You must choose your own topic. The domains below are only samples to help you get started; you are free to pick any other domain, if you can find datasets to use and evaluate your system.

- Education and learning
- Learning management systems (LMS)
- Healthcare
- Legal
- Institute management systems
- Software engineering

4. Research directions

The systems below show the kind of complex, multi-capability agent we expect. These ideas might not be novel. They are examples to inspire you, not a list to choose from. Whatever you build, you still need a clear research question: building the system alone is not enough.

A. Adaptive tutoring team

A team of agents supports a student across several sessions. A diagnostic agent works out why an answer is wrong, a Socratic agent gives hints instead of answers, a planner decides what to study next, and a critic checks each explanation before it reaches the student.

- The system keeps a persistent model of the student's knowledge and updates it after every interaction.
- It retrieves course material and cites the relevant lecture section.
- A small, fine-tuned model could classify error types (conceptual, procedural, careless).
- Possible research question: does role separation with a persistent student model improve remediation compared with a single tutor agent under the same token budget?

B. Agentic viva voce tutor

A voice-based agent conducts an oral examination (viva) on a student's own submitted work, such as a project report, code or assignment. It speaks with the student using speech-to-text and text-to-speech, asks an opening question, and then adapts its follow-ups to probe how deeply the student understands what they submitted.

- It retrieves from the student's submission and the course material so that questions are grounded in what the student wrote.
- It reasons over each answer to decide whether to go deeper, ask for a justification, give a hint, or move on, and scores responses against a rubric.
- It can also run in practice mode as a tutor, giving feedback that helps students prepare for a real viva.

C. Patient-facing care coordination agent

A patient-facing agent handles appointment management, intake and ongoing communication for a clinic. It books, reschedules and cancels appointments, collects symptoms and history before a visit, and sends follow-ups such as reminders or post-visit instructions. Synthetic Longitudinal Patient records can be generated with a tool such as [Synthea](#).

- It uses tools to check clinician availability, book slots, update the patient record and send messages.
- During intake it asks adaptive follow-up questions, retrieves the patient's history and clinic policies, and writes a structured summary for the clinician.
- It must recognize urgent or out-of-scope situations, such as red-flag symptoms or requests for medical advice and escalate to a human instead of answering.
- Possible research question: does an agent that grounds each turn in the patient's record and an explicit escalation policy complete more tasks correctly, and miss fewer urgent cases than a general-purpose chat agent with the same tools?

D. Legal document drafting and review agent

The agent drafts and reviews agreements such as contracts or NDAs. It adapts clause language to the jurisdiction, the counterparty's history and the client's risk profile, then runs the draft through several stages of quality control before a lawyer sees it.

- It retrieves clause templates, jurisdiction-specific rules and past agreements with the same counterparty to produce a first draft.
- Separate reviewer agents check the draft against firm standards, recent precedents and client requirements, and flag inconsistencies such as mismatched definitions or conflicting clauses.
- It compares versions, explains each revision in plain language, highlights strategic shifts such as a liability cap moving in the counterparty's favour, and keeps a history of how each clause has evolved.
- Possible research question: does a multi-stage review pipeline catch more planted inconsistencies and risky terms, with fewer false alarms, than a single pass review by the same model?

E. Incident response agent

Given a task such as "Checkout errors spiked after the latest deployment. Diagnose the problem and propose a recovery plan", the agent investigates a service outage in a sandboxed microservice application. It forms hypotheses, queries logs, metrics, traces and recent deployments, and then recommends or executes approved recovery actions.

- It retrieves relevant runbook steps and past postmortems and cites the evidence behind each conclusion.
- A verifier agent rejects hypotheses that the evidence does not support.
- State-changing actions such as rollbacks or restarts run only after they pass an allow-list and an approval step.

- Possible research question: does a hypothesis-driven investigation loop with evidence checking find the root cause more often, and with fewer unsafe actions, than a standard tool-using agent?

You may take inspiration from these, combine ideas, or propose something entirely different. Keep the scope narrow: one course, one clinical guideline, one contract type, one service or one workflow is enough.

5. Compute constraint: keep it small

The contribution should come from system design, reasoning, data construction, retrieval, memory, tool design, agent architecture or evaluation, not from training a huge model.

- Prefer small/open models, API models, or frozen pretrained models.
- Fine-tuning means LoRA, QLoRA or similar lightweight methods on a small model. Large-scale pretraining is not expected.
- Use small, carefully designed evaluation sets rather than massive datasets.
- A strong evaluation and a convincing research question matter more than a large model.

It is good to include a compute budget estimate in your proposal: the GPU type and hours for any fine-tuning, the expected number of API calls or tokens and their cost, and how many experiment runs you plan. TAs may reject a proposal if you do not show that you have enough compute budget to run the experiments it describes.

6. Proposal and TA review

Propose your idea to your assigned TA before writing any code. TAs will be assigned soon.

- Do not start implementation before your idea is approved. Ideas may be accepted, rejected, or sent back for refinement.
- The proposal must explain the domain, problem, research question, the capabilities you will use, approach, expected novelty, evaluation plan, data and safety plan, and approximate compute.
- TAs may ask you to narrow the scope, change the research question, rebalance the capability mix, strengthen the novelty, or redesign the evaluation.
- The review checks that the project is research-oriented, feasible within the available compute and time, safe for its domain, and distinct enough to support a potential workshop-level submission.
- You may take an idea from this document, combine directions, or propose your own in any domain that fits the overall theme.

7. Project structure and evaluation

Suggested structure

1. **Problem:** one specific problem in your domain, and why existing LLM, RAG or agent approaches do not solve it well.
2. **Research question:** one or two questions that can be tested.
3. **Hypothesis:** what you expect your method to improve, and on which metric.
4. **Method:** the agent architecture, retrieval or memory design, reasoning strategy, tool set, fine-tuning setup and decision policy.
5. **Prototype:** a small working system, with a sandbox or mock tools where actions are involved.
6. **Evaluation:** compare against at least one simple baseline on a controlled dataset, scripted users or injected scenarios.
7. **Analysis:** successes and failures; ablations encouraged.
8. **Write-up:** problem, related work, method, experiments, results, limitations and future work.

Minimum evaluation expectations

- At least one baseline. The baselines can range from existing systems to plain language models without any harness
- A small but carefully designed evaluation set with known ground truth.
- Quantitative metrics wherever possible, including at least one per capability you use (Section 2).
- An ablation of each capability your research question depends on.
- For tool-using or acting systems: tool-call accuracy and an unsafe or unapproved action rate.
- Qualitative examples for difficult or interesting cases.
- Failure analysis: show where the system does not work.
- Do not rely only on "the demo looks good" or a single LLM-as-a-judge score. If you use an LLM judge, validate it against human labels on a sample.

8. What the final proposal must contain

- Project title
- Domain and target user
- Problem statement (5 to 8 sentences)
- Research question
- Why the problem matters in this domain
- What is missing in existing approaches
- Which of the five capabilities you will use, and why your problem needs each one
- Proposed idea and architecture (a diagram is encouraged)
- Expected novelty
- Dataset or evaluation environment
- Baselines
- Metrics, including capability-specific and safety metrics
- Compute plan (GPU hours, API budget)
- Expected risks and fallback plan